ARM Pipelining — Exam Ready Notes

Source: Module 5 “The ARM Instruction Set” pp.22–35. Highlighted italic lines are **comments**.

1. What pipelining is + the ARM7 3-stage pipeline

The ARM uses a pipeline to **increase the speed of the flow of instructions** to the processor: several operations are undertaken **simultaneously** rather than serially, so throughput rises without a faster clock.

- **Fetch** — read the opcode from memory into the internal instruction register.
- **Decode** — activate the appropriate control lines depending on the opcode; decode the registers used and prepare the data-path control signals for the **next** cycle.
- **Execute** — read register(s) from the register bank; shift + ALU operation; write register(s) back to the register bank (for LDR/STR the ALU computes the memory address).

	c1	c2	c3	c4	c5
Instr 1 (at PC)	F	D	E		
Instr 2 (at PC+4)		F	D	E	
Instr 3 (at PC+8)			F	D	E

Key fact (module p.23): the PC points to the instruction being **fetched**, not the one being executed. At any cycle: executing = PC–8, decoding = PC–4, fetching = PC ⇒ the stored PC reads **`PC = current instruction + 8`** in ARM state (**PC+4** in Thumb).

Comment: “Why PC+8?” is the favourite question here. Answer in one line: 3-stage pipeline ⇒ fetch is two stages ahead of execute, and ARM instructions are 4 bytes, so 2 × 4 = 8.

2. Throughput & the pipeline cost formula

- **Simple instructions** (e.g. **ADD**) complete at a rate of **one per cycle** on ARM7TDMI.
- **More complex: STR needs 2 effective clock cycles (+1 cycle)** for the memory access; every cycle still does a memory access, datapath use and a decode for the next cycle.

Cost formula: CPU time = instruction count × **CPI** × clock period. Ideal pipelined CPI = 1; every stall raises CPI.

Comment: pipelining improves throughput, not the time of one instruction (latency stays 3 or 5 cycles). Examiners love this distinction.

3. The ARM9 5-stage pipeline

- **Fetch** — instruction fetched and placed in the pipeline.
- **Decode** — register operands read; the file has **3 operand read ports**, so most instructions source all operands in one cycle.
- **Execute** — operand shifted and ALU result generated; load/store addresses computed in the ALU.
- **Buffer/Data** — data memory accessed if required, else the ALU result is simply buffered one cycle.
- **Write back** — results, including data loaded from memory, written to the register file.

Item	ARM7	ARM9
Architecture	Von Neumann (one shared bus)	Harvard (separate I & D memory)
Pipelining	3-stage (F–D–E)	5-stage (F–D–E–Buf–WB)
ISA	v4T	v4T

Comment: link the three facts — Harvard → fetch and data access in the same cycle → a deeper 5-stage pipeline is possible. Always quote them together.

4. Pipeline hazards and their fixes (module pp.28–35)

Hazard	Cause	Fix
Structural	Two instructions want the **same resource** in the same cycle (one memory for instr + data).	Separate instruction & data memories/caches (**I$ and D$**); register file: **read in 2nd half** of cycle, **write in 1st half**.
Data	An instruction needs a **result not yet ready** (classic `lw` then `add` using it).	Stall (bubbles), or **forwarding/bypassing** the result.
Control	A **branch** must decide the next PC before the condition is evaluated and the target computed.	Stall (this **hurts CPI**), branch prediction, or ARM’s **conditional execution**.

General rule (module): a hazard can always be resolved by **waiting** (stalling) — but pipeline control must first **detect** the hazard and **take action** to resolve it; stalling costs CPI.

Comment: memorise one example per hazard — single memory (structural), lw→add (data), beq (control). “Fix by waiting but it affects CPI” is the module’s own sentence for the control hazard.

5. One-minute revision

Numbers: ARM7 = 3 stages · ARM9 = 5 stages · both ISA v4T · PC+8 (ARM) / PC+4 (Thumb) · ADD = 1/cycle · STR = 2 cycles · 3 read ports · ideal CPI = 1.

Likely questions: (1) Explain the ARM instruction pipeline with a diagram. (2) Why does PC read as PC+8? (3) Compare ARM7 and ARM9 pipelining. (4) Define the three hazard types and give a fix for each. (5) How does pipelining increase throughput?

Comment: always draw the F–D–E staircase with PC, PC–4, PC–8 labels (section 1) — it is the diagram that scores full marks.